



中南大學
CENTRAL SOUTH UNIVERSITY

计算机视觉前沿技术

CSE
计算机学院

多层感知机

主讲： 沈成超

邮箱： scc.cs@csu.edu.cn

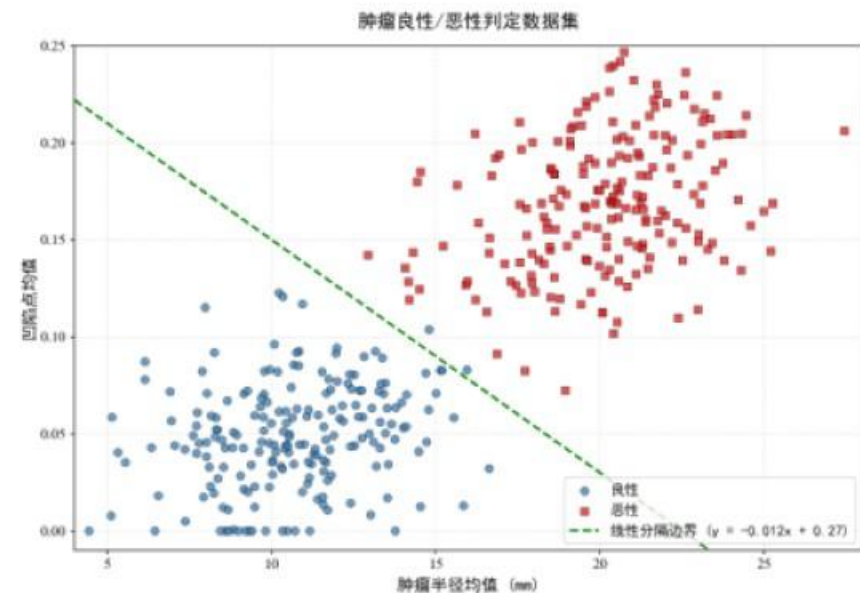
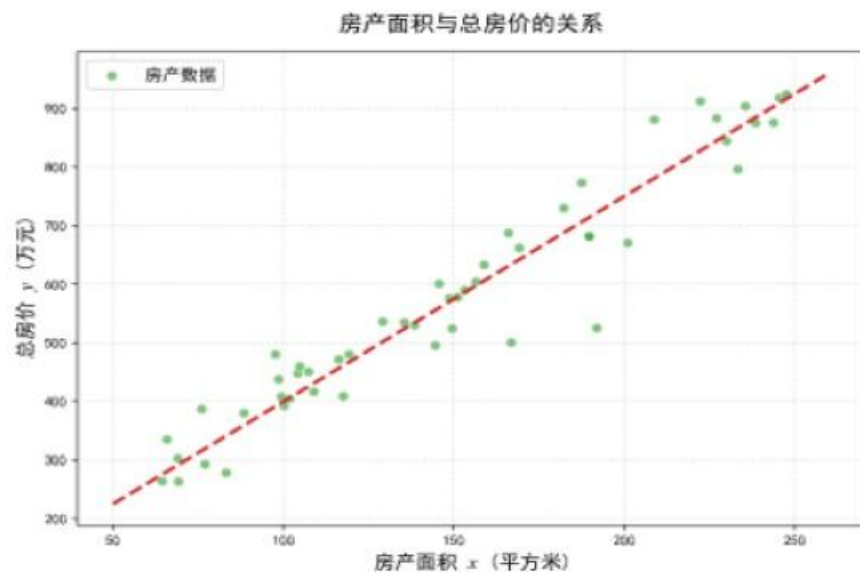
2026.03.20

目录



- 1. 线性网络的局限**
- 2. 非线性问题**
- 3. MLP的特性**
- 4. MLP的训练**
- 5. MLP的训练技巧**
- 6. MLP的应用**
- 7. 课程作业**

1. 线性网络的局限：简单回顾



线性网络：

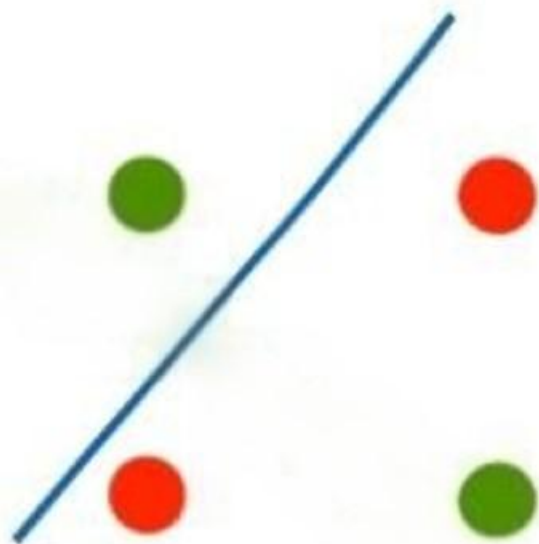
无论组合形式，数学表达式始终是加权求和

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

优势：

简单、优雅
能很好地解决线性问题

1. 线性网络的局限:



对异或 (XOR) 问题束手无策

线性网络:

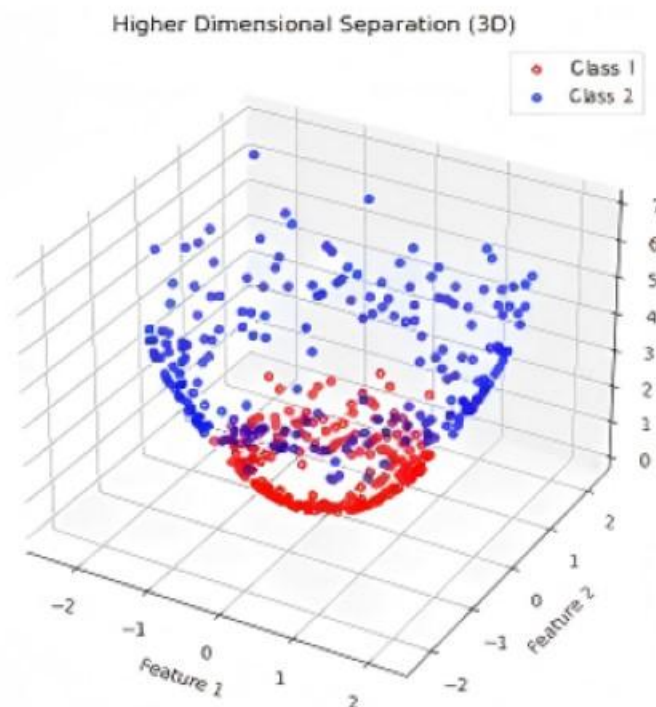
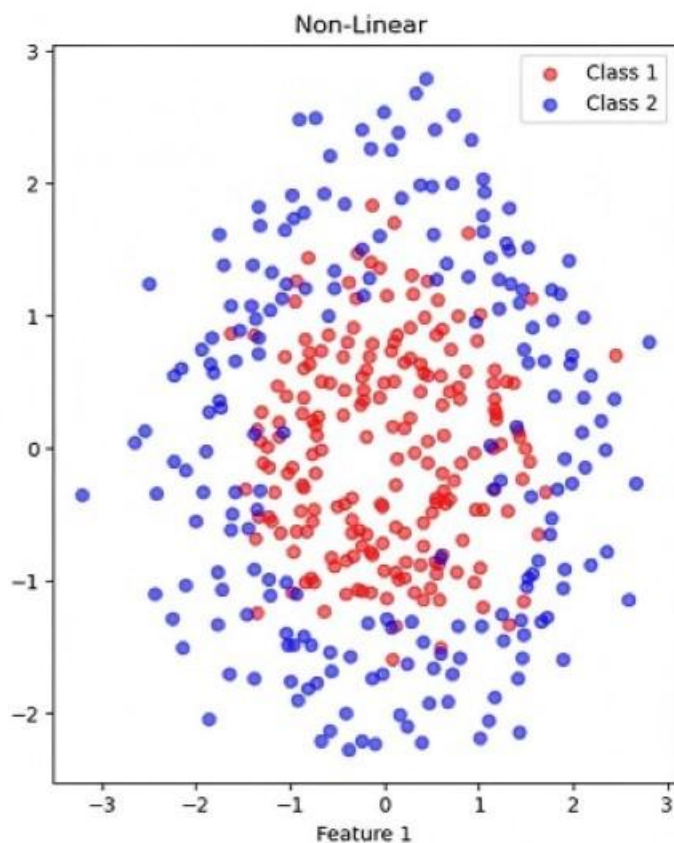
无论组合形式, 数学表达式始终是加权求和

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

多层线性网络叠加, 数学上等价于单层网络

表达能力弱, 只能处理线性可分的问题

2. 非线性问题：非线性特性



- **复杂问题：**现实世界的复杂数据往往是线性不可分的
- **激活函数：**激活函数通过非线性运算，将低维空间的数据“扭曲”并映射到高维空间使得原本纠缠的数据变得线性可分。

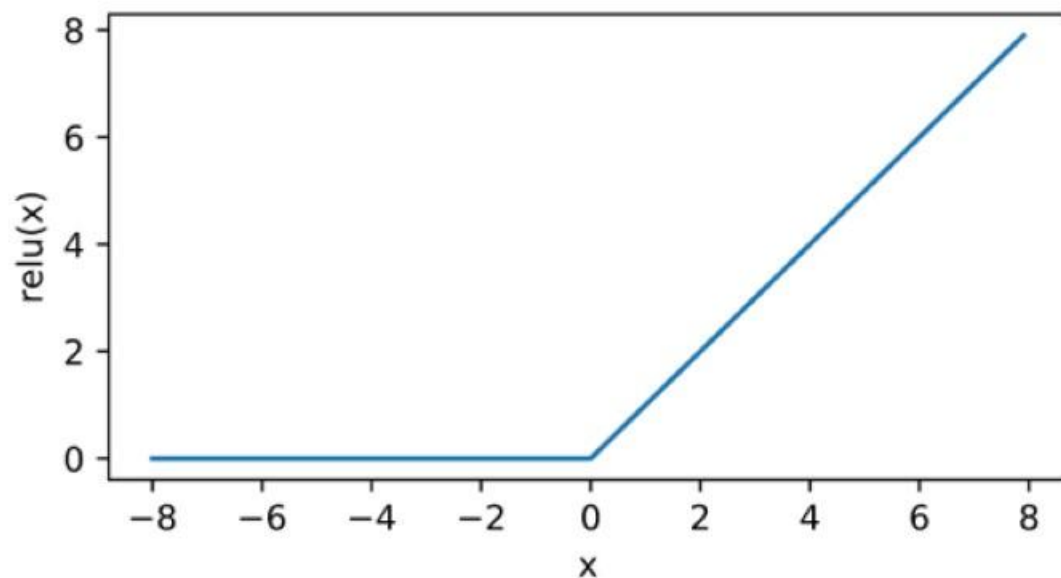
加入非线性特性

2. 非线性问题：非线性激活函数

线性 (ReLU) 修正函数

ReLU: 线性修正单元

$$\text{ReLU}(x) = \max(x, 0)$$

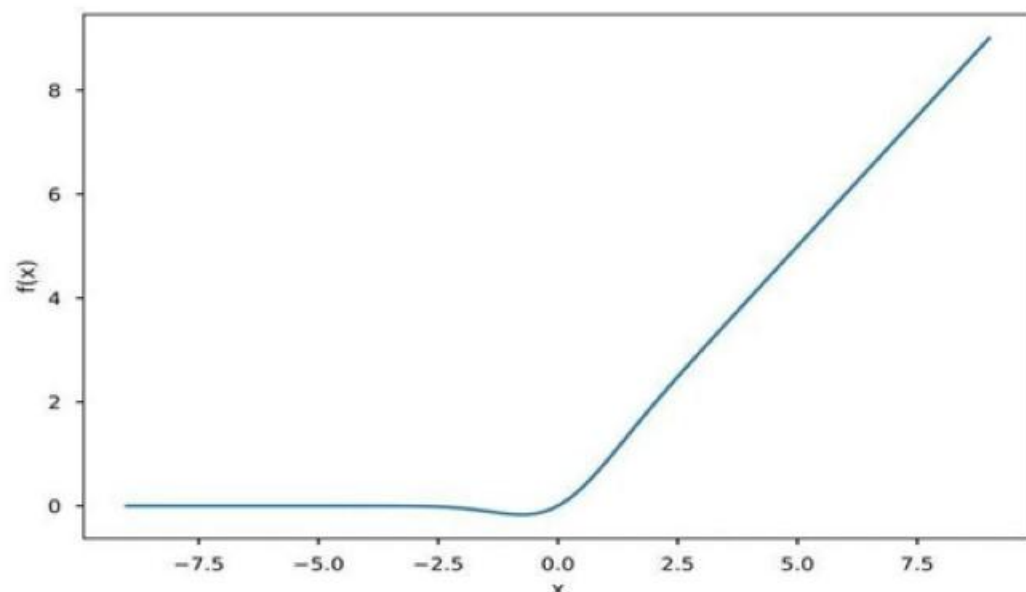


2. 非线性问题：非线性激活函数

高斯误差线性 (GeLU) 激活函数

将输入映射到约 $(-0.17, +\infty)$ ，并在零点附近平滑过渡

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi}(x + 0.044715x^3) \right) \right)$$

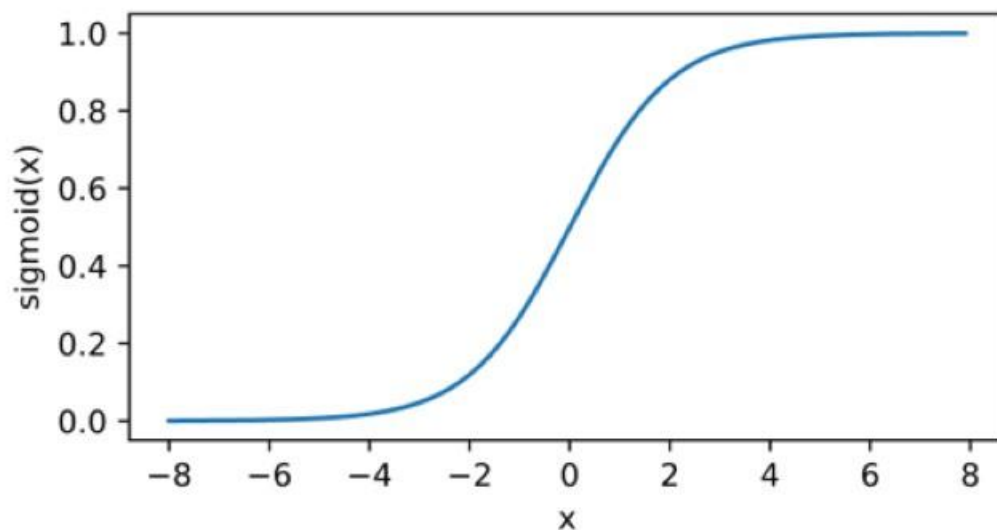


2. 非线性问题：非线性激活函数

S型 (sigmoid) 激活函数

将输入映射到 $(0, 1)$, 一个平滑的版本 $\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$

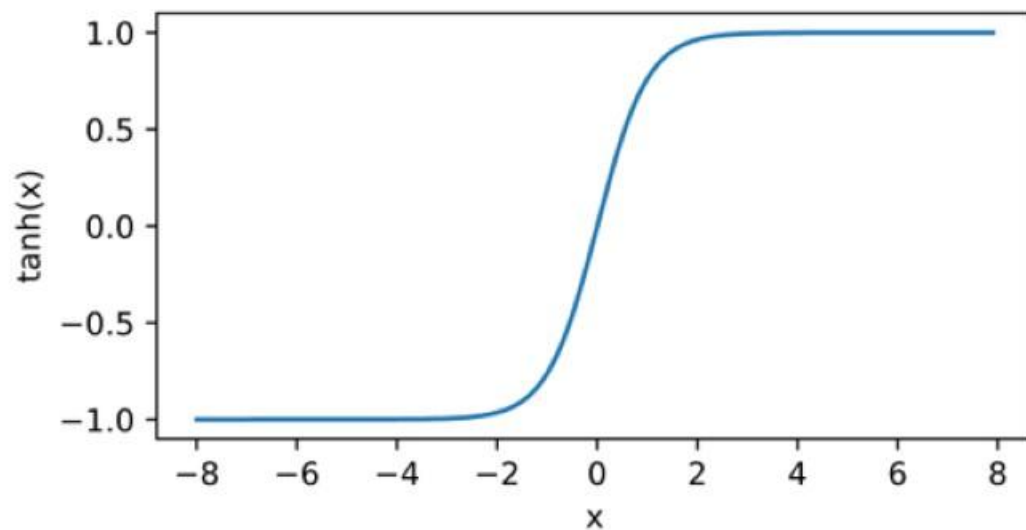


2. 非线性问题：非线性激活函数

双曲正切 (tanh) 激活函数

将输入映射到 $(-1, 1)$

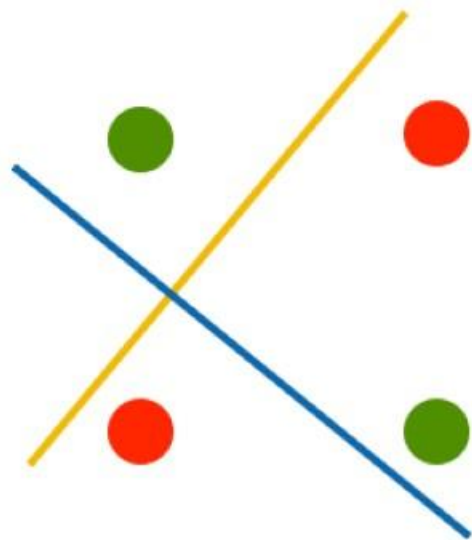
$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



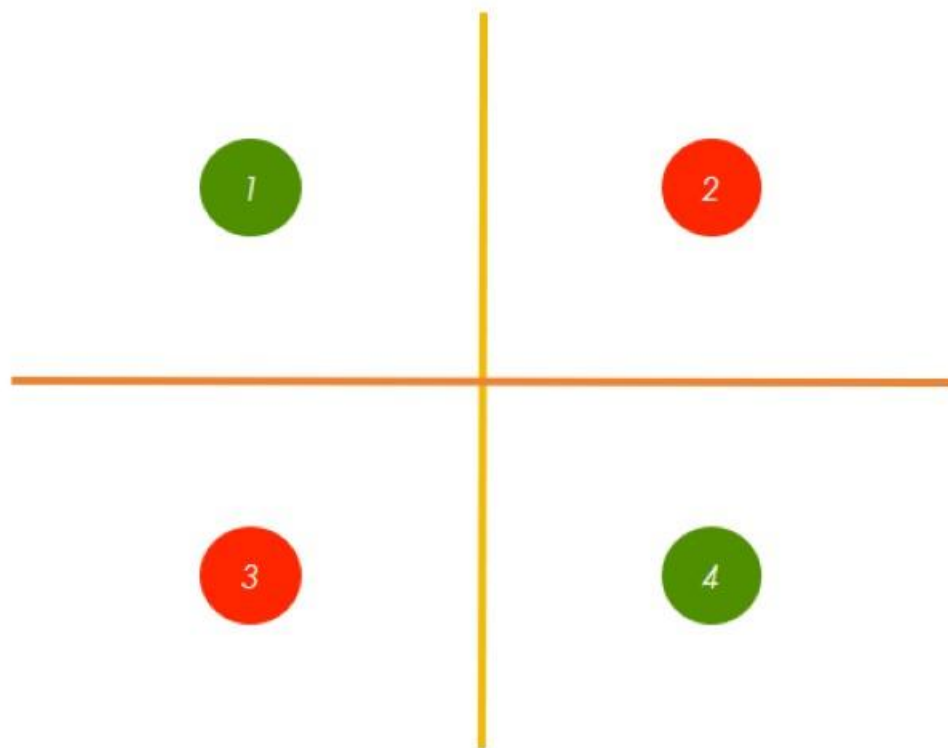
3. MLP的特性：攻克XOR问题

XOR 问题 (Minsky & Papert, 1969)

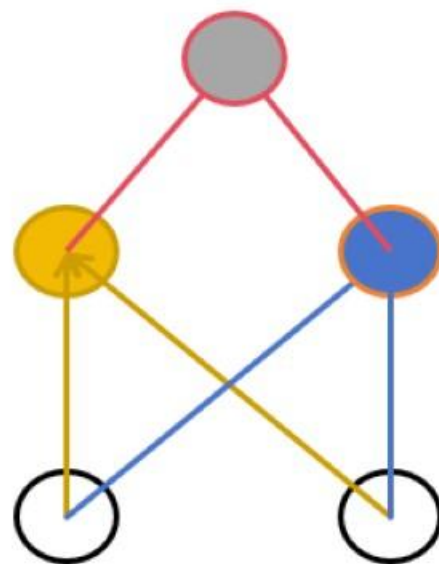
感知机无法学习 XOR 问题（神经元只能生成线性分离器）



3. MLP的特性：学习XOR



	1	2	3	4
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+



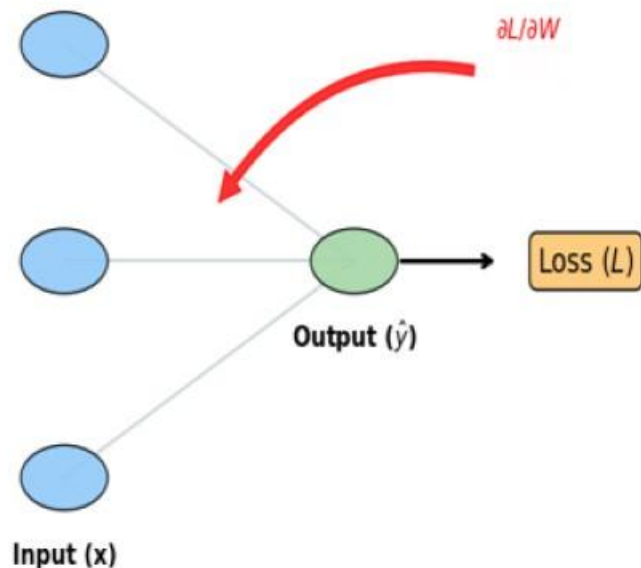
3. MLP的特性：通用近似定理

- 只要网络足够宽，单隐层神经网络可以以任意精度拟合任意复杂的连续函数
- 必须使用非线性激活函数，否则无论多宽多深，依然只是线性变换
- 理论上可以，实际易导致过拟合

4. MLP的训练

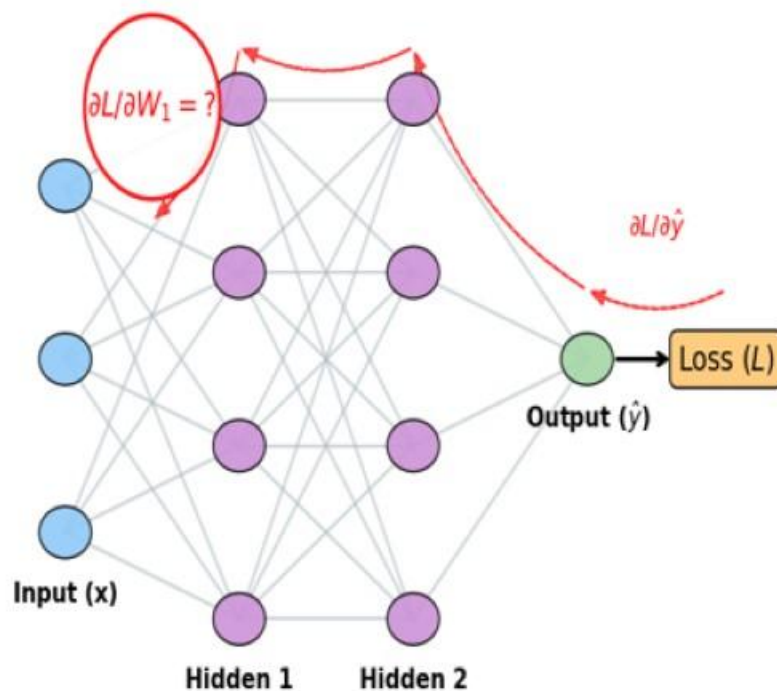
在之前的课程中，关于线性模型，我们通过计算损失函数关于权重的偏导数来更新参数：

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$



然而，在多层网络中，损失函数在最外层，底层的权重深埋在网络内部。

如何求出底层参数的梯度？



4. MLP的训练：反向传播算法

- 链式法则 – 标量：

$$y = f(u), u = g(x)$$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x}$$

- 链式法则 – 矢量：

$$\frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial \mathbf{x}}$$

$$(1, n) \quad (1, 1) \quad (1, n)$$

$$\frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

$$(1, n) \quad (1, k) \quad (k, n)$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

$$(m, n) \quad (m, k) \quad (k, n)$$

4. MLP的训练：反向传播算法

设定第 l 层的线性计算为 $z^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}$, 激活输出为 $h^{(l)} = \sigma(z^{(l)})$

梯度如何传播？

定义第 l 层的“误差信号”为 $\delta^{(l)} = \frac{\partial l}{\partial z^{(l)}}$ 。误差从 $l+1$ 层传回 l 层的公式为：

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

如何更新权重？ $\frac{\partial l}{\partial W^{(l)}} = \delta^{(l)}(h^{(l-1)})^T$

由 $\delta^{(l)}$,可得： $\frac{\partial l}{\partial b^{(l)}} = \delta^{(l)}$

4. MLP的训练：训练循环

```
for epoch in range(epochs):
    for X, Y in dataset:

        # --- 1. 前向传播 (Forward) ---
        H[0] = X
        for l in range(1, L+1):
            Z[l] = W[l] @ H[l-1] + b[l]
            H[l] = sigma(Z[l])      # 激活函数

        Loss = cross_entropy(H[L], Y)

        # --- 2. 反向传播 (Backward) ---
        delta[L] = softmax交叉熵梯度 # 输出层误差
```

```
for l in range(L, 0, -1):
    # 计算当前层梯度
    grad_W[l] = delta[l] @ H[l-1].T
    grad_b[l] = sum(delta[l])

    # 链式法则：将误差回传给上一层
    if l > 1:
        delta[l-1] = (W[l].T @ delta[l]) * sigma_deriv(Z[l-1])

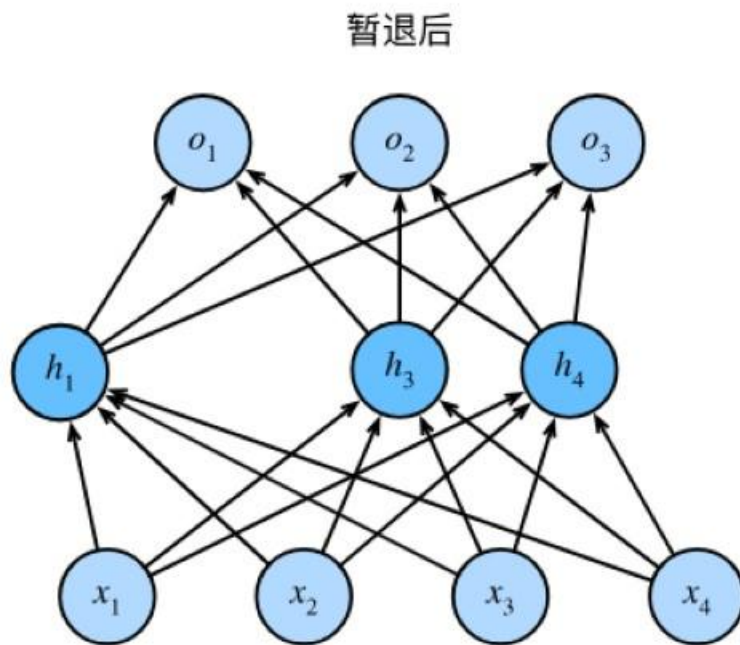
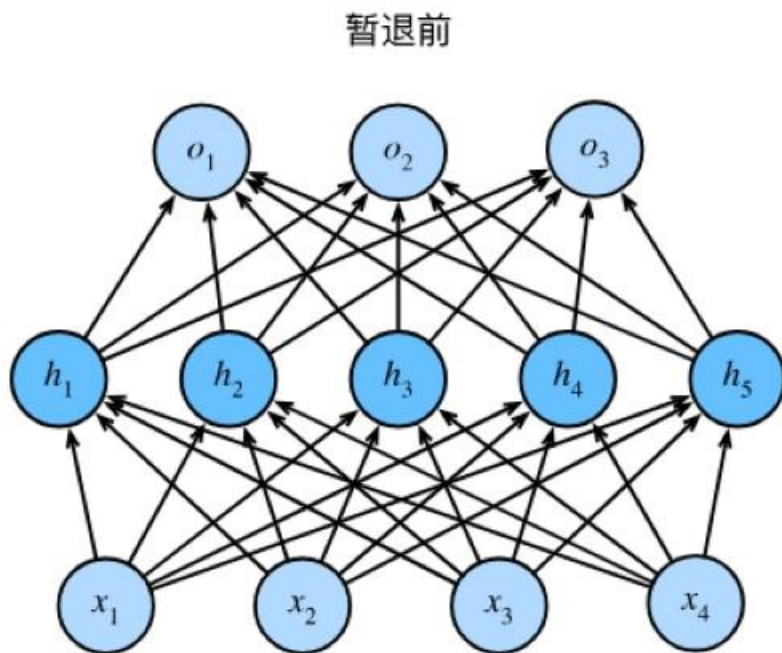
    # --- 3. 参数更新 (Update) ---
    for l in range(1, L+1):
        W[l] -= eta * grad_W[l]   # 梯度下降
        b[l] -= eta * grad_b[l]
```

5. MLP的训练技巧：如何避免MLP过拟合

- Dropout 随机失活神经元

- 根据此模型的设计，其期望值保持不变，即 $E[h'] = h$ 。

$$h' = \begin{cases} 0 & \text{概率为 } p \\ \frac{h}{1-p} & \text{其他情况} \end{cases}$$



5. MLP的训练技巧：如何避免MLP过拟合

- Normalization

将隐藏层输出的特征数据，强制拉回“均值为 0，方差为 1”的标准分布

BN (Batch Norm)

给定一个批次大小为 m 的输入集合 $\beta = \{x_1, x_2, \dots, x_m\}$

计算批次均值与方差 $\mu_\beta = \frac{1}{m} \sum_{i=1}^m x_i, \sigma_\beta^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_\beta)^2$

标准化处理 $\hat{x}_i = \frac{x_i - \mu_\beta}{\sqrt{\sigma_\beta^2 + \varepsilon}}$

可学习的仿射变换 $y_i = \gamma \hat{x}_i + \beta$

5. MLP的训练技巧：如何避免MLP过拟合

- BN依赖 Batch Size, 难以处理长度不一的动态序列数据 (如自然语言文本)

LN (Layer Norm)

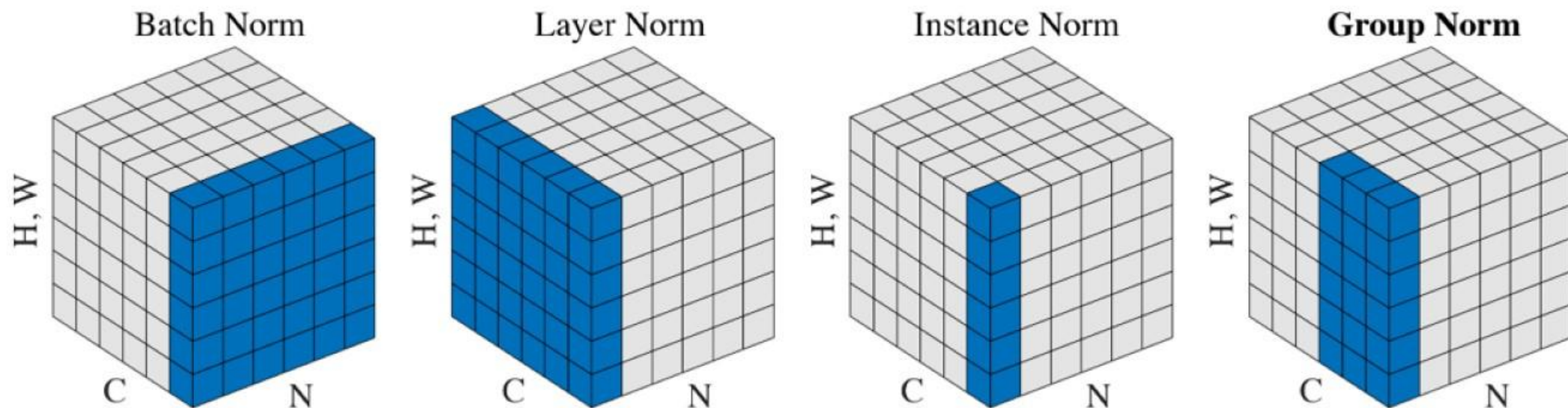
不再跨样本求均值, 而是**对单一样本的所有特征维度**求均值和方差。

针对第 i 个样本, 特征维度为 d

$$\mu_i = \frac{1}{d} \sum_{j=1}^d x_{i,j}, \quad \sigma_i^2 = \frac{1}{d} \sum_{j=1}^d (x_{i,j} - \mu_i)^2$$

5. MLP的训练技巧：如何避免MLP过拟合

Batch Norm 按通道跨样本归一化, Layer Norm 按样本跨通道归一化



5. MLP的训练技巧：如何避免MLP过拟合

➤ 权重衰减：Weight Decay

在原始损失函数中，强行加入一个惩罚项，限制权重 w 不能太大

$$L_{new} = L_{old} + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

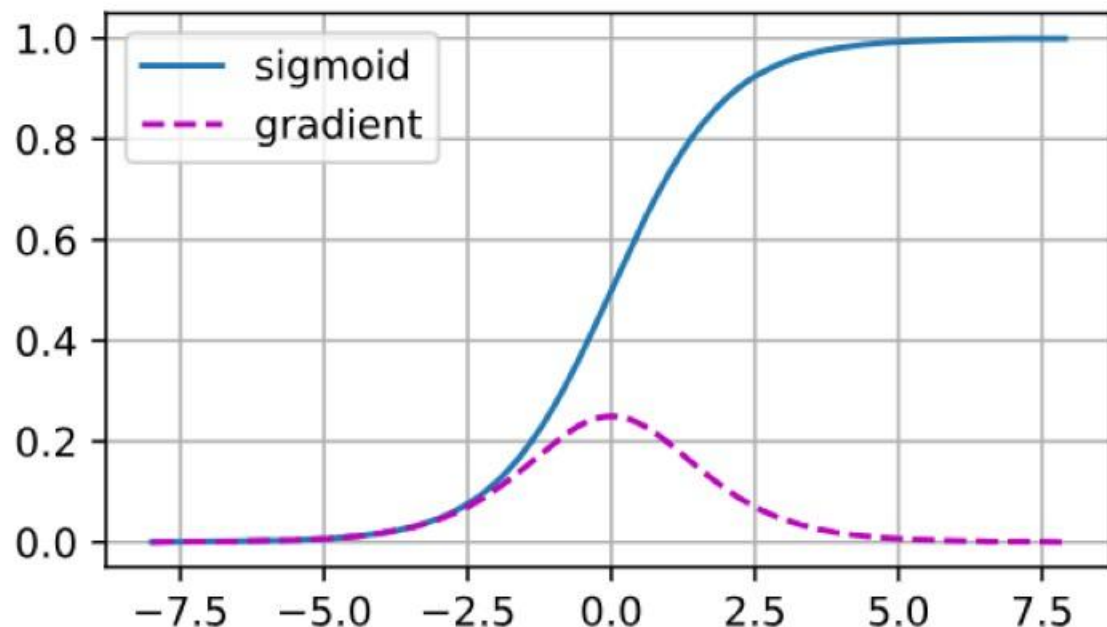
对新的损失函数求导并进行梯度下降更新时：

$$\frac{\partial \tilde{L}}{\partial \mathbf{w}} = \frac{\partial L}{\partial \mathbf{w}} + \lambda \mathbf{w}$$

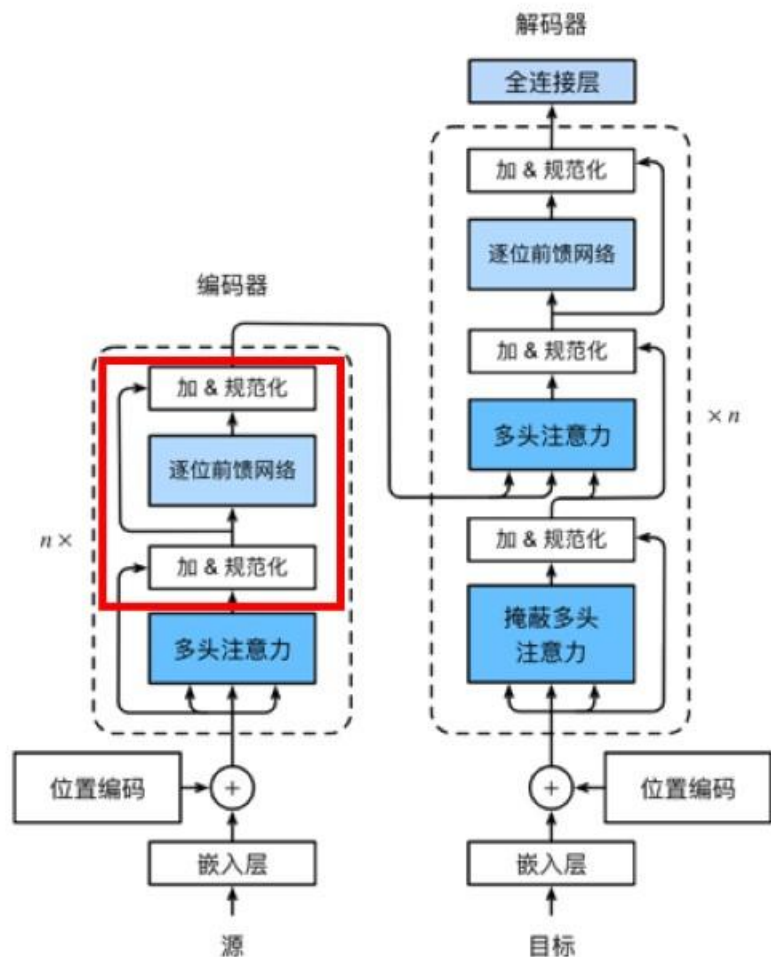
5. MLP的训练技巧：如何避免梯度消失

➤ 梯度消失

反向传播中多个小于 1 的导数项连乘，导致底层权重的更新信号趋近于 0



5. MLP的训练技巧：如何避免梯度消失



- 进行归一化（如前文提到的 LN/BN）将数据分布拉回均值为 0、方差为 1 的基准区间。
- 在现有的神经网络（如 Transformer）中，归一化已经成为了确保梯度流动的基础组件。

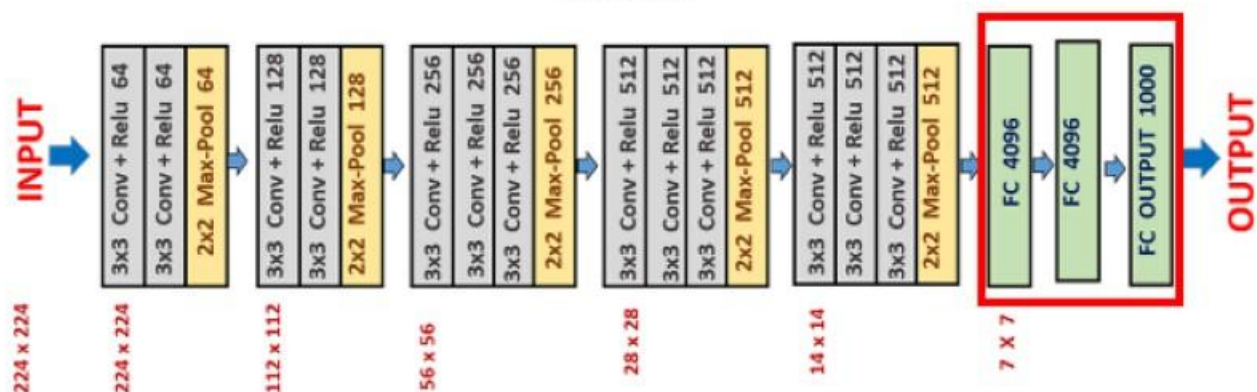
Transformer 中的 MLP (Feed Forward) 模块，一般采用2层

- 第1层：将输入特征极度拉宽，配合非线性激活函数（如 GELU）在宽阔的高维空间中完成复杂的特征提取
- 第2层：将特征重新降维投射回 d 维输出

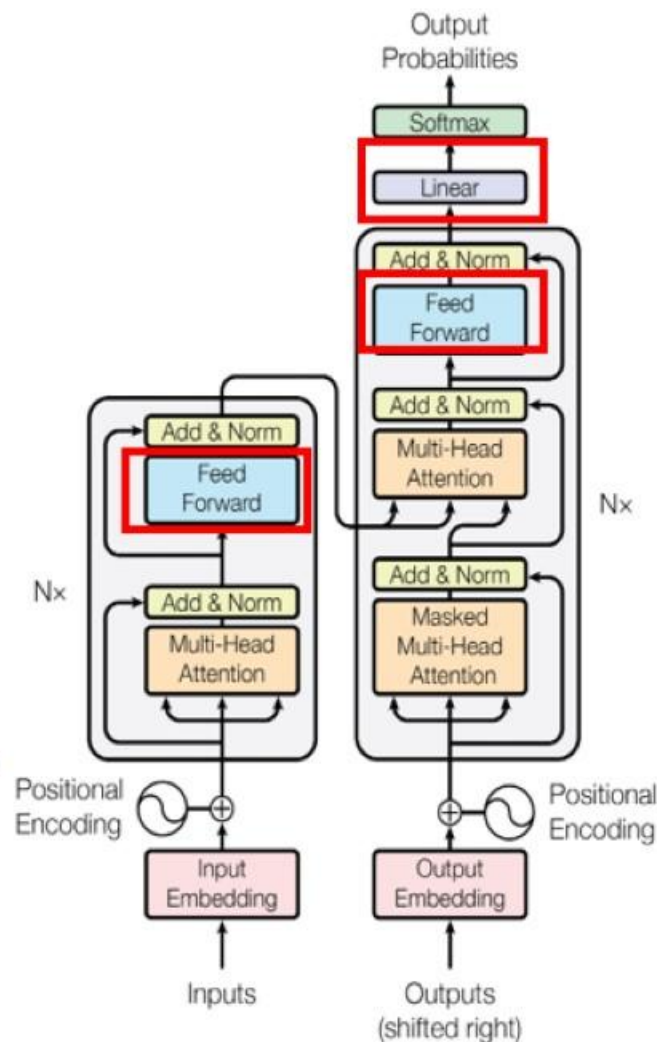
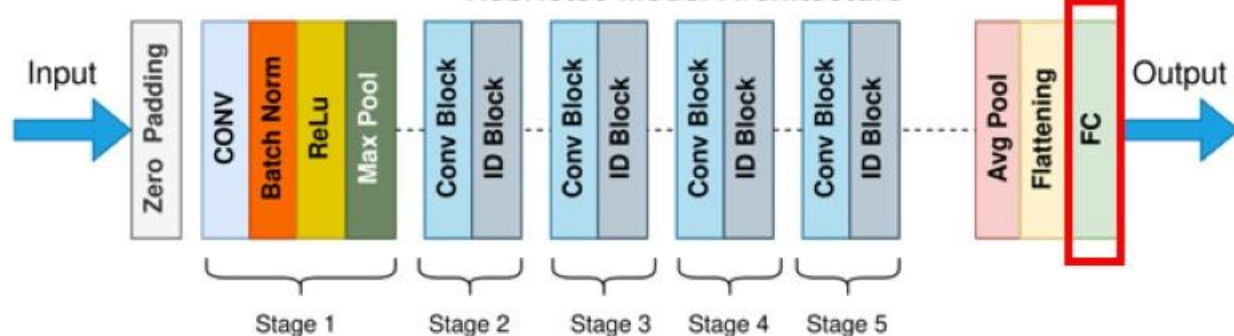
6. MLP的应用

MLP在现代各种顶尖网络架构中作为不可或缺的“非线性推理引擎”

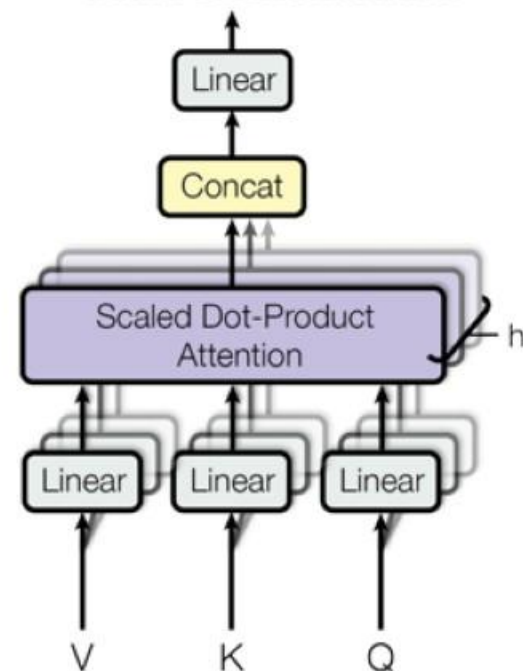
VGG-16



ResNet50 Model Architecture



Multi-Head Attention



7. 课程作业

用Numpy实现MLP进行Mnist分类

- 数据集读取及预处理，采样每个类的前30%作为训练集（CPU训练慢）
- 线性层（2层）
- 非线性激活函数，ReLU
- 损失函数
- 梯度计算+反向传播
- 模型参数更新
- 损失函数，验证集准确率曲线